Feedback based on your works:

1. "It runs on my machine" is not a result - finish the run before writing results.
This project's own log file proves the point: full_run.log cuts off mid-dataset-build, and there's no pidm_output/ folder anywhere in the repo. It's tempting to draft the results section from the *expected* numbers in your design doc while the actual run is "still going in the background." Don't. If your thesis or paper reports a number, you should be able to point to the exact CSV/log line it came from. Teach students to treat "experiment completed successfully end-to-end, artifacts saved" as its own checklist item - separate from "code compiles."

2. Reproducibility seeding - do this well, and do it everywhere.
This is genuinely a strength worth highlighting: random_state=42 / seed=42 is used consistently across dataset sampling, splits, and graph layout (real_data_loader.py, synthetic_generator.py, scale_study.py, baseline_comparator.py). That's good practice - a reviewer or a fellow student should be able to rerun your pipeline and get the same split. The one gap: I don't see torch.manual_seed() set anywhere before the DeBERTa Trainer is initialized in classifier.py. Model *training* is the one place where nondeterminism actually changes your headline metric. Tell students: seed your data pipeline **and** your model init/training, or your F1 score is only reproducible up to noise.

3. Auto-tuning hardware config is nice engineering, but document what changed.
config.py's VRAM-based auto-selection (DeBERTa-base vs. small vs. DistilBERT depending on GPU) is a genuinely useful pattern for teams with mixed hardware. But it means the *model that actually got trained* depends on whose machine ran it. For a paper, this must be logged explicitly in the methods section - "trained on an RTX 3070 (8GB), which selected DeBERTa-v3-base with batch_size=16, grad_accum=2, fp16" - not left implicit in a config file. Any auto-adaptive step in your pipeline is a reproducibility risk unless you log the resolved values, not just the logic.

4. Pin your dependencies before you submit, not after.
requirements.txt uses open-ended version bounds (torch>=2.9, transformers>=4.44, unconstrained datasets, sentence-transformers, etc.). This is fine for active development, but if a reviewer or your examiner tries to rerun this in six months, pip install -r requirements.txt may pull a newer transformers release with breaking API changes (the code already has defensive inspect.signature handling around TrainingArguments - a sign

this has already bitten you once). Rule for students: freeze exact versions (pip freeze > requirements.txt) for the version of the codebase tied to your submitted results.

5. Statistical rigor isn't optional at Q1 level - build it in from day one, not at revision time.
Across the whole pidm/eval/ module there's no McNemar's test, no bootstrap confidence intervals, no significance testing of any kind - despite having a real held-out test set and six baseline systems that make paired comparison trivial to set up. This is the single most common reason ML papers bounce at review in security/detection venues. Teach it as a design-time requirement: **before** running your final experiment, write down which statistical test will validate "PIDM beats baseline X," not after a reviewer asks for it.

6. Hardcoded hyperparameters need either a search or a citation - "we chose these" isn't a method.
The ensemble weights (rbf: 0.20, classifier: 0.45, sid: 0.20, gcpd: 0.15) and all four detection thresholds are fixed constants with a comment saying "tune via ablation" - but no ablation exists yet in the code. This is a very common thesis pattern: a plausible-looking number gets hardcoded early for a demo and never revisited. Have students flag every hardcoded numeric constant in their own code and ask: "is this justified by data, or is it a guess I forgot to go back to?"

7. Match your proposal to your implementation - reviewers cross-check.
The Proposal.docx targets a 2,000–5,000 message dataset; the actual code and README target ~40,000. Small inconsistencies like this are exactly what makes a proposal/thesis/paper feel unpolished even when the underlying work is solid. Before submission, do a deliberate pass comparing every number in your write-up against the number in your actual code/config.

8. Cite the benchmarks your reviewers will already know.
The lit review cites five foundational papers but misses the standard comparators in this specific subfield (InjecAgent, AgentDojo, OWASP LLM Top 10). This is a broader lesson: for any subfield, there's a "reviewer's checklist" of 3–5 papers/benchmarks that anyone working in the area is expected to know and position against. Missing them doesn't just cost citations - it signals to a reviewer that you may not know the space as well as your results claim.

9. Watch for evaluation leakage in anything with memory/state.
The graph-based cascade detector accumulates a running suspicion score per agent across the message stream. If train and test messages aren't strictly separated by conversation

trace when this state is built, information can leak across the split. Any component with memory (running averages, graph state, session context) needs an explicit "reset at test time" check - this is a subtle bug class that's easy to introduce and easy to miss without deliberately testing for it.